WAC
Windows Artefacts Collector

CENTRE D'EXCELLENCE CYBERDEFENSE AEROSPATIALE

Created by the Aerospace Cyber Defence Centre of Excellence (CEC)

Against KAPE and EZ

WAC (Windows Artefacts Collector)

Windows artefact collection software for digital forensic investigation



SYSTEM ARTEFACTS

USER FILES

DIGITAL EVIDENCE

A TOOL IN SUPPORT OF DIGITAL INVESTIGATION

WAC — Windows Artefacts Collector

Against KAPE and Eric Zimmerman's tools

Software version	1.4.0
Document edition	October 2026
Publisher	Centre d'Excellence Cyberdéfense Aérospatiale (CEC)
	École de l'air et de l'espace, air base 701, Salon-de-Provence

This document confronts WAC with the community's reference tools: KAPE for collection, Eric Zimmerman's command-line tools for decoding. On the same exhibits of the same machine, it compares running times, the accuracy of the values — every difference settled by an independent reference — and the output formats. It says what the confrontation taught WAC, and what it still holds against it.

Contents

1 The method **2**

1.1 What is compared 2

1.2 Who is right when they differ 2

1.3 The test bench 2

2 Summary **4**

3 Running times **6**

3.1 Decoding 6

3.2 Collection 6

4 Accuracy **8**

4.1 In favour of WAC 8

4.2 In favour of the EZ tools 9

5 Output formats **10**

5.1 What each tool writes 10

5.2 What sets them apart 10

6 What the confrontation taught WAC **12**

7 Limits of the comparison **13**

CHAPTER 1

The method

1.1 What is compared

Two questions, two comparisons.

Collection: WAC against KAPE
Both tools collect the same machine one after the other; what each copies is compared, the time it takes, and the SHA-1 of the files both copy.

Decoding: WAC against the EZ tools
On the *same exhibits* — the exhibit store of a WAC collection —, WAC converts an artefact and the matching Eric Zimmerman tool decodes it; both outputs are compared record by record and field by field.

1.2 Who is right when they differ

A difference between two tools does not say which one is wrong. Every difference was therefore settled by a third source, independent of both:

- Windows itself: the XML it renders for an event (`Get-WinEvent`), a file's date, the NTFS object identifier (`fsutil objectid`), the live registry, a file's hash (`Get-FileHash`);
- another implementation: `dissect.esedb` (Fox-IT) for ESE databases;
- the format itself: sequence numbers and Marvin32 digests of the hive logs, read byte by byte;
- two artefacts of the same machine cross-checked: Prefetch for execution, a row's timestamp for its other dates.

Each difference is then classed: *defect of WAC* (fixed), *error of the EZ tool* (checked), or *convention* of either — two forms of one value.

1.3 The test bench

Machine	Windows 11 Pro (build 26200), virtual machine
EZ tools	version 2026.5.0 (.NET 9.0.20)
KAPE	version 1.3.0.2, target `KapeTriage`
WAC	version 1.4.0 with the fixes this confrontation brought

Everything can be replayed: `vmtest/ez-compare.sh <artefact>` times WAC and the EZ tool on the same exhibits then compares their outputs; `vmtest/kape-compare.sh` does the same for

collection. The EZ tools ran with their full date precision (`--mp`, or `--dt` with seven decimals where `--mp` is not enough), so as not to blame them for a loss of precision a setting avoids.

CHAPTER 2

Summary

Artefact	Volume	Result	Errors of the EZ tool, checked
$MFT — MFTECmd	362,081 records	paths, states, 8 dates identical	sizes of 1,490 WOF files and 3 open logs: Windows gives WAC's
EVTX — EvtxECmd	190,613 events	system fields identical; values explained	6 wrong values (Windows); 335 Perflib events unreadable; cleared logs ignored
Prefetch — PECmd	379 files	all identical	hash without its leading zero (19)
Shortcuts — LECmd	3 files	identical to 100 ns	—
Jump lists — JLECmd	13 shortcuts	identical to 100 ns	—
Shellbags — SBECmd	27 folders	identical	wrong NodeSlot at first level (7); HasExplored always false
Amcache — AmcacheParser	1,258 entries	124 columns identical	2,340 invented values; partial SHA-1 not flagged
ShimCache — AppCompat-CacheParser	232 entries	identical	misleading execution flag
Recycle bin — RBCmd	19 files	identical	—
SRUM — SrumECmd	60,804 rows	identical	17 wrong energy rows (dissect)
Registry — RECmd	9 artefacts	identical	4 hives rolled back to an older state
Collection — KAPE	1,130 common files	SHA-1 identical	—

Key point

On everything both sides read, WAC gives the same values as the reference tools, or more accurate ones, each time established by an independent source. The confrontation also revealed 23 defects of WAC, all fixed (chapter 6), and coverage gaps, opened for follow-up (section 4.2).

CHAPTER 3

Running times

3.1 Decoding

Same machine, same exhibits; each tool handles its artefact only.

Artefact	WAC	EZ tool	What WAC does besides
$MFT (380 MB)	7.3 s	8.1 s	two volumes (C: and G:) against one
EVTX (305 logs)	16.8 s	22.0 s	plain-text message rebuilt for every event
Registry (Kroll batch)	2.0 s	7.6 to 13.6 s	every hive replayed, every registry artefact
SRUM (21 MB)	3.2 s	2.5 s	2 more tables, identities resolved
Prefetch (380)	3.6 s	0.7 s	dated local time, paths with drive letter
Shortcuts, jump lists	1.8 s	0.3 s	
Recycle bin	1.9 s	0.5 s	
Amcache, ShimCache, shellbags	2.0 s	0.5 to 1.4 s	one pass for all three (registry key)

On the large artefacts, WAC is faster. On the small ones, the EZ tools are: every WAC conversion pays a fixed cost of 1.5 to 3.4 s, measured with a trivial artefact. That cost is the verified working copy of the hives, the replay of their logs, and the reading of the suspect's time zone and profiles. It is what lets WAC date every local time with the offset in force at that date, and give the drive letter; the EZ tools give neither.

3.2 Collection

Tool, setting	Time	Exhibits	Volume
KAPE, `KapeTriage`	40 s	1,198	0.91 GB
WAC, kit, no binaries	197 s	2,463	1.63 GB
WAC, kit, no event logs	18 s	817	—

Tool, setting	Time	Exhibits	Volume
WAC, full kit (`binary: unverified`)	454 s	61,294	3.6 GB

On a comparable scope, KAPE collects five times faster. Almost all of the gap comes from the event logs (180 s): WAC adds the 1,225 message provider binaries (680 MB) that rebuild every event's text offline. That time, spent on the examined machine, is to be reduced (item B-033). The full kit also checks the Authenticode signature of every executable on the machine and copies those whose signature does not hold, which KAPE does not do.

CHAPTER 4

Accuracy

4.1 In favour of WAC

Hive logs replayed in order
The `.LOG1` and `.LOG2` logs keep, behind the recent entries, leftovers of an earlier generation. RECmd applies them in file order, hence after the newer ones. It thus rolls SYSTEM, SECURITY, DEFAULT and a service profile's hive back to an older state: in SYSTEM, sequence 0x3618 instead of 0x3620, and four last BAM executions shown 13 minutes early. WAC sorts the entries by sequence and stops at the break in the chain. The logs themselves confirm it, Marvin32 digests included.

Events of cleared logs
A cleared log keeps its records: WAC reads them again and flags them `EvtRecovered`, never presenting them as live; EvtxECmd does not see them.

Event values as Windows renders them
Hexadecimal `0x3e7`, GUIDs in braces, booleans `true`, binary `69060000`, dates with their `Z`, U+200E marks kept: the forms Event Viewer shows and one searches for. EvtxECmd changes eight of them, and is wrong on six values, such as a boolean `False` that Windows renders `true`.

Nothing invented
A value absent from the source is not emitted. AmcacheParser writes 2,340 default values absent from the hive (`IsPeFile: False` for a PE executable), SrumECmd 7,086 (0 or -1).

Partial Amcache SHA-1 flagged
For a file larger than 30 MiB, Amcache hashes only the first 31,457,280 bytes (measured on `mrt.exe`). WAC says so (`Sha1CoversBytes`); without it, comparing with the whole file's SHA-1 would wrongly conclude to a modification.

No misleading field
The ShimCache "executed" flag, which AppCompatCacheParser gives, says "no" for 105 of the 117 binaries that Prefetch proves were run. WAC does not collect it: that "no" would lead to conclude a program never ran.

Precision to 100 ns
WAC gives every date at its source's precision. By default, PECmd, LECmd and SrumECmd round to the second, and LECmd ignores `--mp` in its CSV.

Dated local time
Every date is given in UTC and in the suspect's local time, with the offset in force at that date. The EZ tools give UTC only, without saying so by a `Z`.

Others' readings checked
SBECmd's "FirstInteracted" and "LastInteracted", RECmd's "CreatedOn" of an account, are

readings of key write times. WAC publishes those times themselves: anyone can draw their own reading from them, without inheriting a ready-made one. SBECmd also gives a wrong `NodeSlot` (0) to first-level folders.

SrumECmd's ESE reading errors
17 energy rows out of 887 carry the content of another row or an identifier absent from the table; dissect.esedb sides with WAC.

Collection
Each WAC exhibit carries MD5, SHA-1 and SHA-256, the eight NTFS dates of its original and its MFT entry, in a sealed manifest. KAPE gives one SHA-1 and three dates.

4.2 In favour of the EZ tools

Registry coverage
RECmd's Kroll batch reads keys WAC ignores: installed programs, Defender exclusions, RecentDocs, MountPoints2, known networks, the NTFS last-access update setting, shutdown time, among others. Item B-031.

File coverage
KapeTriage copies files WAC does not take: shortcuts outside Recent (Start menu, Desktop, taskbar), the Default profile's hive, `$Secure:$SDS` and other NTFS metadata, Defender traces (item B-032); browsers and the Windows timeline, already planned (B-016, B-010).

Speed on small artefacts and in collection
See the previous chapter.

Ready-to-read interpretations
The EZ tools split and name some fields: executable, date and description of a SRUM application, known folder names, a file's extension. WAC gives the raw value, as Windows stores it.

> **Limit**
> What WAC does not collect, it cannot weigh: KAPE's coverage remains wider on browsers and several registry keys, until B-016, B-031 and B-032 are done.

CHAPTER 5

Output formats

5.1 What each tool writes

Tool	Formats	Note
AmcacheParser	CSV	8 files, one per table
AppCompatCacheParser	CSV	
RBCmd	CSV	
SrumECmd	CSV	7 files
EvtxECmd	CSV, JSON, XML	
JLECmd	CSV, JSON, HTML	
LECmd	CSV, JSON, HTML, XML	
MFTECmd	CSV, JSON, bodyfile	
PECmd	CSV, JSON, HTML	
RECmd	CSV, JSON	plus one CSV per plugin
SBECmd	CSV, JSON	
KAPE	copy of the files, CSV copy log	
WAC	JSON for every artefact	one file per artefact, plus the sealed manifest

5.2 What sets them apart

Four tools out of eleven write CSV only. In the others, the JSON transcribes the CSV line: one text line per record (JSON Lines), flat.

Types
In the EZ tools, numbers are strings: a run count is written `"RunCount":"458"`, a record number `"EventRecordId":"21572"`. WAC writes numbers, booleans, strings.

Structures
In the EZ tools, a list becomes a comma-joined string (the files a Prefetch loaded, its directories), and an event's data are JSON escaped *inside* a string (`Payload`), to decode a second time. WAC nests: a Prefetch's volumes and their directories, a shortcut's items, an event's `Name`/`Value` pairs.

Dates
In the EZ tools, UTC is not flagged (`2026-09-30 23:04:46`) and precision depends on an option. WAC gives full ISO 8601, in UTC (…`Z`) and in local time with its offset (…`+02:00`).

Missing value
The EZ tools write a default value, or an empty cell. WAC omits the field; an artefact not collected is declared so (`NotRequested`, `NotCollected`), so that an empty list is not taken for an absence of traces.

Provenance
WAC's manifest ties every value to the exhibit it comes from, with its hashes and original dates.

Console
The EZ tools detail their work on the console; some write the decoding itself there when no file is asked for. WAC writes only its progress there: everything it finds is in the files.

> **Key point**
> For an analyst, the difference shows at the first processing: a WAC JSON loads as it is into an analysis tool or into WAE, with nothing to split again; an EZ output is handled like a spreadsheet, whose numbers must be converted back, lists split again and event data decoded again.

CHAPTER 6

What the confrontation taught WAC

Each defect below produced a valid JSON that was wrong, or incomplete. All are fixed, with a check that would have caught them.

Artefact	Defect fixed
$MFT	stale extension records merged with the base
EVTX	floats cut to 6 digits; binary as “69 06 00 00”; hexadecimal integers and GUIDs in the wrong form; ProcessingErrorData ignored (21 events without data); events of cleared logs not read
Shortcuts	URI glued to the block after it; reading 2 bytes off without an ID list; ExtraData blocks not read (machine, tracking); target size not read; null DROIDs emitted
Jump lists	open count and MRU rank not read; DestList format version 1 not supported; application name corrupted
Shellbags	folder lost with everything below it; the folder’s key time, rank and NodeSlot missing; “Home” folder without a name
Amcache	2 tables out of 14 and 4 to 6 values read: no SHA-1, size, driver or device
ShimCache	tabs of packaged applications replaced by spaces
SRUM	FILETIME dates left as numbers; UTF-16 text read as ANSI
Registry	BAM limited to accounts with a profile (a deleted account lost its executions); UserAssist foreground time not read
Test harness	a crashing check silently stopped every following one

CHAPTER 7

Limits of the comparison

- One machine, virtual, running Windows 11: some artefacts are few there (3 shortcuts in Recent, 2 USB keys). The formats are the same on a real machine, but not every rare case is represented.
- The two collections follow each other instead of being simultaneous: the files written between them ($MFT, logs, Prefetch) rightly differ.
- The EZ tools ran with their maximum precision options; by default, they lose more.
- The comparison covers the common artefacts. What only one tool reads has no reference here: its accuracy rests on the checks of the validation report.